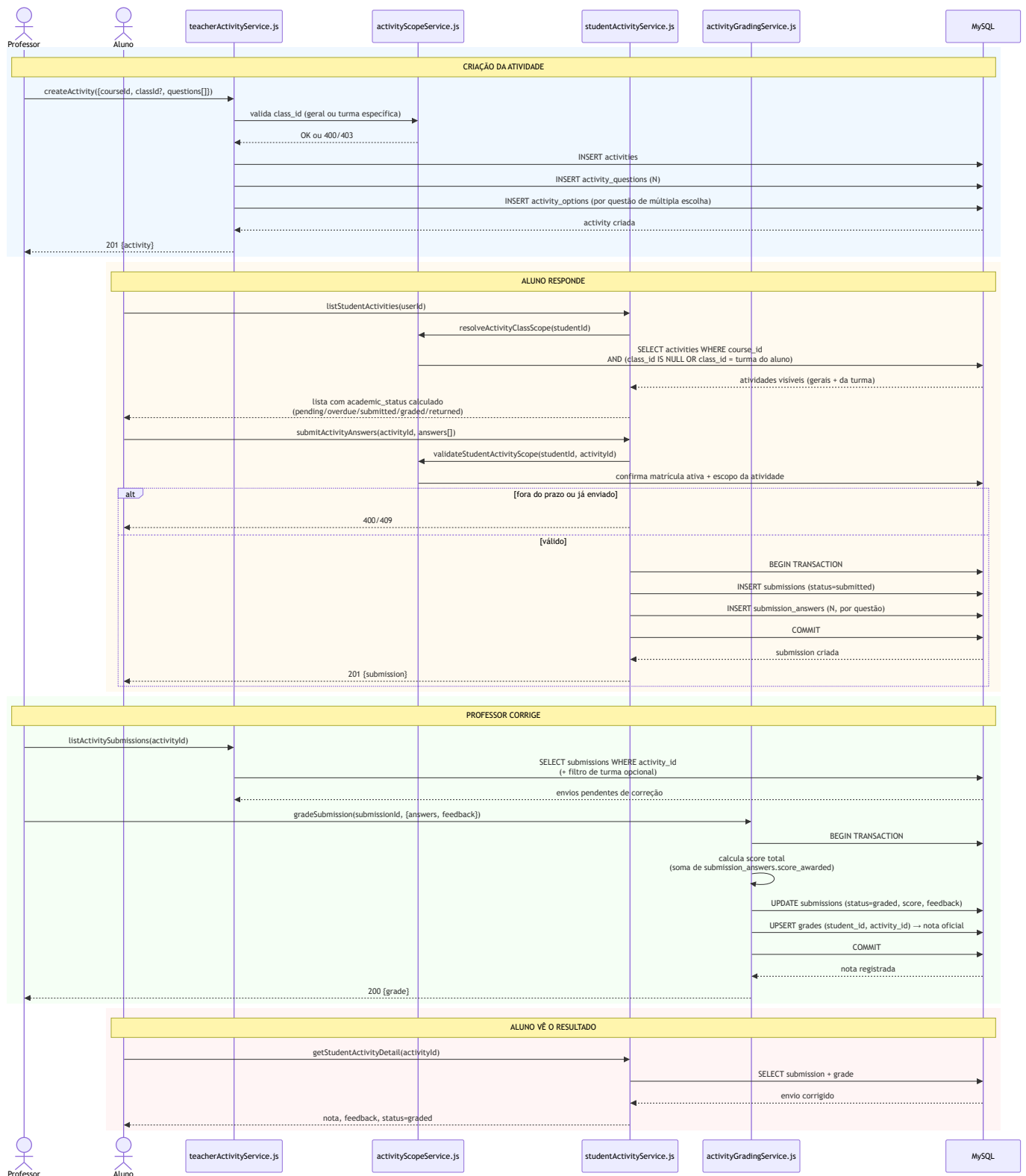


Diagrama — Fluxo de Atividades e Avaliações

Cobre o ciclo completo: criação pelo professor, resposta pelo aluno, correção pelo professor.

Nomes reais: `teacherActivityService.js` , `studentActivityService.js` ,
`activityGradingService.js` , `activityScopeService.js` .



Que problema este diagrama esclarece

Atividades e avaliações compartilham a mesma tabela (`activity_kind`) e o mesmo pipeline de envio/correção, mas isso não é óbvio para quem só olha as telas do frontend (`AtividadesAluno.jsx` e `AvaliacoesAluno.jsx` parecem produtos separados). Este diagrama deixa explícito que é um único

fluxo de dados, com uma única regra de negócio central: escopo geral-vs-turma, aplicado tanto na criação quanto na listagem.

Como interpretar os elementos

- Os quatro blocos coloridos são fases sequenciais no tempo de vida de **uma** atividade — da criação até o aluno ver o resultado — não fluxos paralelos.
- `activityScopeService.js` aparece em dois pontos diferentes (criação e envio) porque a mesma regra de escopo (`class_id` `NULL` vs. específico) precisa ser respeitada em ambas as pontas: o professor não pode criar uma atividade para uma turma que não é sua, e o aluno só pode enviar para atividades visíveis no seu próprio escopo.
- A transação na correção (`gradeSubmission`) atualiza duas tabelas na mesma operação atômica (`submissions` e `grades`) — o diagrama mostra isso explicitamente porque é fácil presumir, incorretamente, que só uma tabela guarda o resultado da correção.

Que decisões arquiteturais este diagrama evidencia

- **Um envio por aluno por atividade, sem histórico de versões:** o diagrama não mostra um "reenvio" porque não existe no fluxo real — a restrição `UNIQUE(student_id, activity_id)` em `submissions` significa que corrigir substitui o estado existente, não cria um novo registro.
- **A nota "oficial" (`grades`) é derivada do envio, nunca editada independentemente:** não existe um caminho no diagrama onde um professor edita `grades` sem passar por `gradeSubmission` sobre uma `submission` real — reforça que a nota sempre tem proveniência rastreável a um envio específico.
- **Bloqueio de edição de questões após existirem envios** (não mostrado neste diagrama de fluxo temporal, documentado em `business-rules.md` seção 4) é uma decisão que só faz sentido à luz deste diagrama: mudar as questões depois que `submissions / submission_answers` já existem quebraria a integridade histórica do que foi de fato respondido.

Trade-offs que este fluxo representa

- **Cálculo de status acadêmico em tempo de consulta** (`pending / overdue / submitted / graded / returned`), nunca armazenado: garante que o status está sempre correto em relação ao relógio atual (um item nunca fica "pending" para sempre depois que o prazo passa, por exemplo), em troca de uma comparação de data em toda consulta de listagem — aceitável dado o volume de dados do domínio, mas seria um ponto a revisar se o número de atividades por curso crescesse ordens de grandeza.

- **Escopo de turma resolvido a cada chamada, não cacheado:** cada listagem de atividades para um aluno refaz a resolução "qual é a turma deste aluno neste curso" — simples e sempre correto, mas significa que um aluno com múltiplas matrículas ativas paga esse custo de resolução uma vez por curso a cada carregamento de tela.
- **Feedback e nota vivem na mesma operação de correção:** um professor não pode salvar feedback textual sem também finalizar a nota (não há "correção parcial salva como rascunho" no fluxo de `gradeSubmission`) — simplicidade de implementação em troca de menos flexibilidade para o professor que quer revisar em etapas.

Como este diagrama deve evoluir

Se o produto ganhar suporte a reenvio (aluno corrige e reenvia após `feedback returned`), este diagrama precisa de um quinto bloco mostrando esse ciclo — hoje `returned` é um estado terminal na prática, sem um caminho de volta para `submitted` documentado em código. Se a correção ganhar granularidade por questão (nota parcial salva antes de finalizar), atualize o bloco de correção para refletir os passos intermediários.